1
Getting Started with Spring AI

As software engineers, we want to create software with powerful features that excite users in unexpected ways. We even want our peers to wonder, how did we do that?

From the very beginning, Spring has focused on developer productivity by removing complexity and adhering to object-oriented fundamentals in a well-structured and intuitive manner. Spring has made simple things easy and difficult things possible. Spring has evolved to encompass all application concerns, from data access to messaging and integration patterns, batch processing, observability, security, cloud computing, and now, AI engineering.

Now, Artificial Intelligence, or AI for short, has the potential to make what was once science fiction, possible. AI will breathe new life into applications in unforeseen ways. Indeed, the best kind of technology is one that is only felt, but never seen.

A simple metric for success is the **Turing Test**, "*a test of a machine's ability to exhibit intelligent behavior equivalent to, or indistinguishable from, that of a human.*" See https://en.wikipedia.org/wiki/Turing_test. Only when technology seamlessly blends into everyday life, adapting to its environment in an instant, will truly intelligent applications be realized.

Spring has been at the forefront of innovation, oftentimes setting the standard by which everything else is measured. With AI, it is no different, and the Spring team has once again set the bar in the Java community. Therefore, when you combine the maturity of Spring with the power of AI, you arrive at something truly special.

Welcome to Spring AI.

In this chapter, you will build your first Spring AI application. You will learn how to switch between different AI providers and models, and how to run your application locally. Specifically, we will cover the following topics:

- From `start.spring.io` to `spring-boot:run` in 2 minutes
- Switching AI Provider Models in a Single Step

- Think Global, Build & Run Local

Enough talk. Let's code!

Free Benefits with Your Book

Your purchase includes a free PDF copy of this book along with other exclusive benefits. Check the *Free Benefits with Your Book* section in the Preface to unlock them instantly and maximize your learning experience.

Technical requirements

All examples for this book are kept in the GitHub repository, https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI.

A `README` is provided for each example, explaining the runtime requirements and instructions to run the example from the command line, using Maven, and Spring Boot. Alternatively, you can import and run each example inside your IDE. Your IDE can configure the project based on the provided Maven POM.

The Maven POM is already configured to resolve the binary version of Spring Boot and Spring AI needed to run the examples. You only need to download and install JDK 17, or newer, on your computer. You may use either Oracle JDK or OpenJDK.

To find the latest information on Spring AI, open a web browser and navigate to https://spring.io/projects/spring-ai/. Click on the **LEARN** tab. All current and supported versions are listed along with references to documentation and Javadoc.

From start.spring.io to Spring Boot: Run in two minutes

To get started quickly, we will build a really simple Spring AI application to prompt an AI and output its generated response.

Like the start to any good Spring project, you start a Spring AI project by going to **Spring Initializr** at https://start.spring.io. You might find a link to **Spring Initializr** by going into your bookmarks and searching through your top 5.

After you land at Spring Initializr, select your language [**Java**, **Kotlin**, **Groovy**] and project [**Maven**, **Gradle**] of choice. I recommend you use the latest **generally available**

(GA) version of Spring Boot, which is `3.5.0` at the time of this writing. Spring Boot `3.x` minimally requires Java 17.

We will bundle the application in a JAR file with a dependency on OpenAI. Click on **ADD DEPENDENCIES** and type in **AI** or **OpenAI**. In the list that appears, select **OpenAI**:

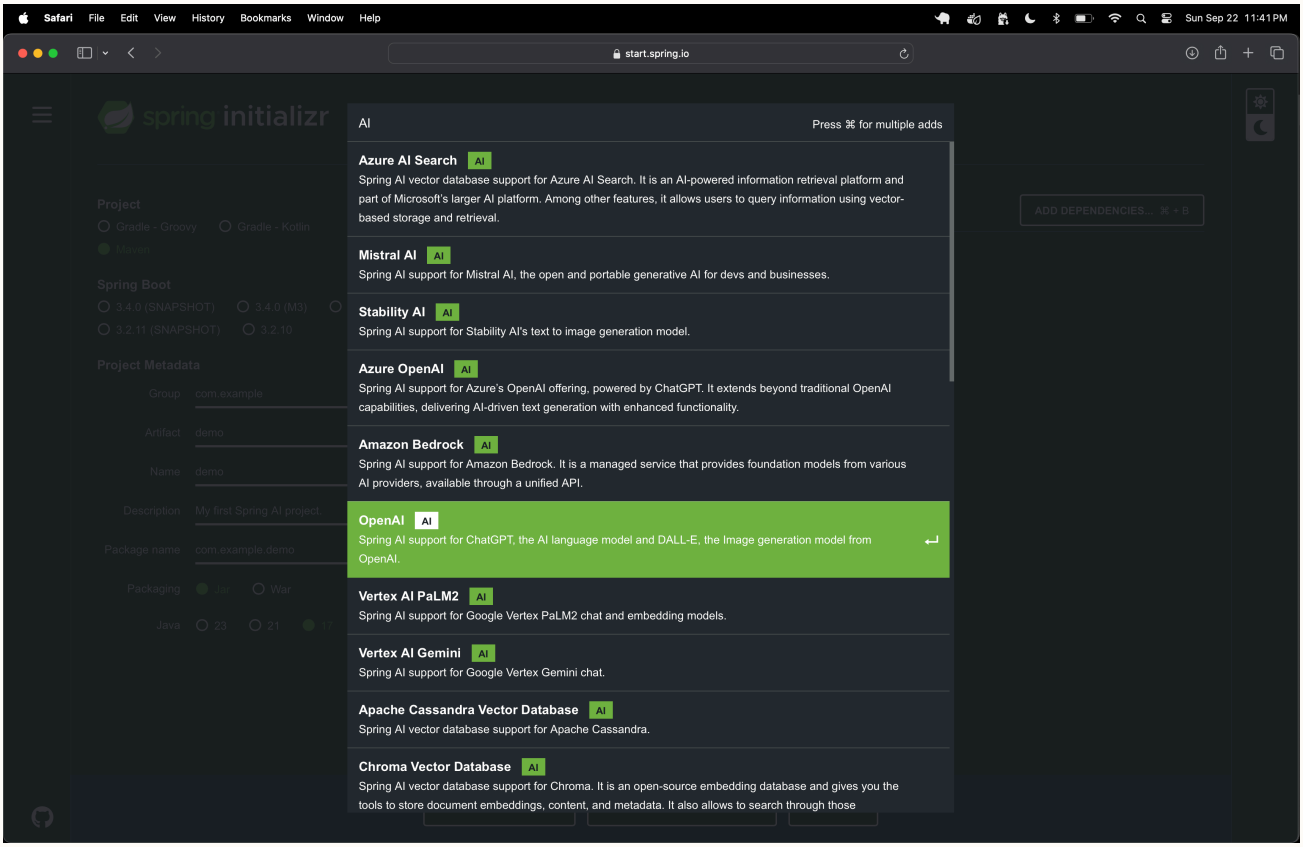


Figure 1.1: Adding dependencies with Spring Initializer

If you simply typed AI, you will see that there are many AI providers supported by Spring AI to choose from. We will come back to this topic in a bit.

The end result should look similar to this:

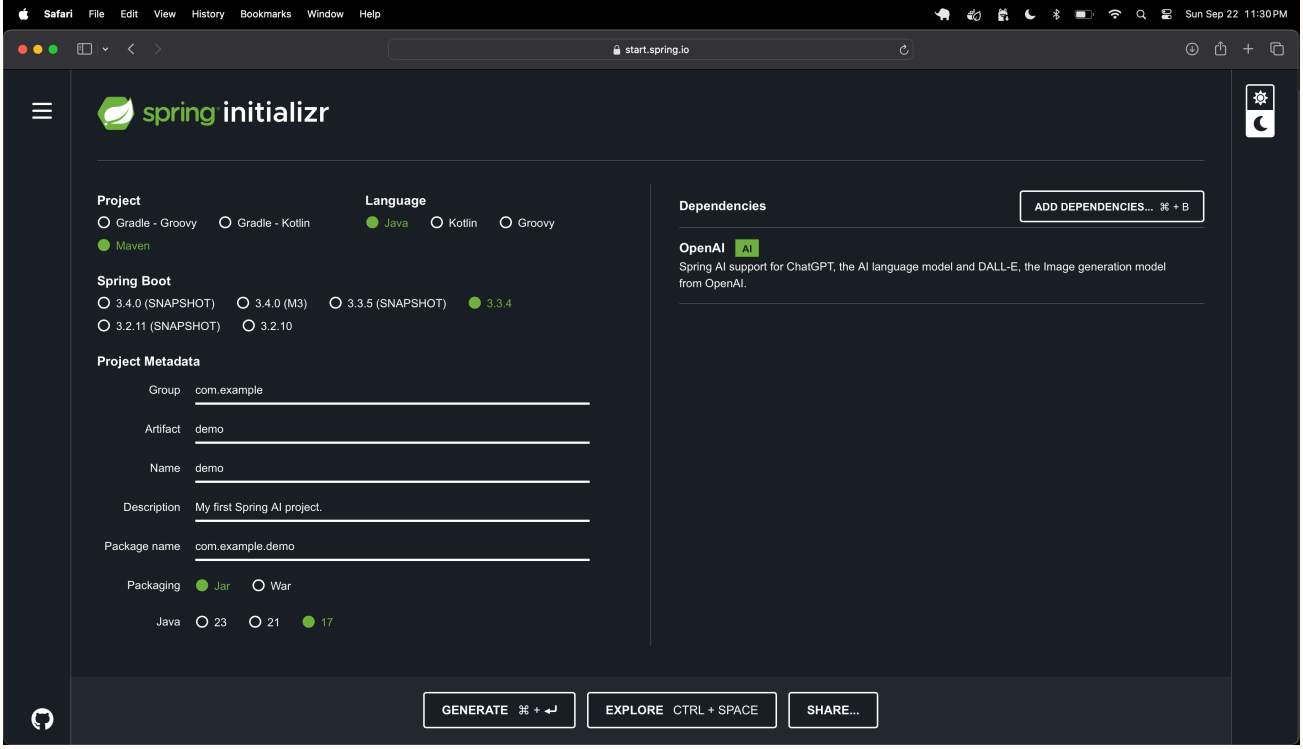


Figure 1.2: Generating a Spring project

Now, you can click on **GENERATE** to create the project.

You can also preview the contents of the Maven `pom.xml` or `build.gradle[.kts]` file by clicking on the **EXPLORE** button before generating the project.

Some of the key elements in the Maven POM file are listed as follows:

```
<properties>
  <java.version>17</java.version>
  <spring-ai.version>1.0.0</spring-ai.version>
</properties>
```

You can see from the properties that we are using JDK 17 and Spring AI `1.0.0`. The `spring-ai.version` property is used in the Maven `dependencyManagement` block to control the version of all Spring AI dependencies declared and used in your Spring Boot application.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>${spring-ai.version}</version>
      <type>pom</type>
```

```
    <scope>import</scope>
   </dependency>
  </dependencies>
</dependencyManagement>
```

When OpenAI was added to the project, **Spring Initializr** declared `spring-ai-starter-model-openai` in the project's dependencies.

```
<dependencies>
 <dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>
   spring-ai-starter-model-openai
  </artifactId>
 </dependency>
 <!-- additional dependencies omitted -->
</dependencies>
```

By declaring the Spring AI BOM using Maven's `dependencyManagement` block, it is not necessary to declare a version for any Spring AI dependency. See the complete Maven POM (https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/blob/main/example-one/pom.xml) in the GitHub repository for this example.

Gradle users can achieve the same outcome as Maven. Take a look at the following Gradle build file, shown here, in Kotlin:

```
plugins {
 java
 id("org.springframework.boot") version "3.5.0"
 id("io.spring.dependency-management") version "1.1.7"
}
java {
 toolchain {
  languageVersion = JavaLanguageVersion.of(17)
 }
}
extra["spring-ai.version"] = "1.0.0"
dependencyManagement {
 imports {
  mavenBom("org.springframework.ai:spring-ai-bom:${property("spring-ai.version")}")
 }
}
```

```
dependencies {
  implementation("org.springframework.ai:spring-ai-starter-model-openai")
}
```

As shown in the preceding code, the `build.gradle.kts` file generated by **Spring Initializr** is very similar to the Maven POM. It includes dependency management, a dependency declaration on OpenAI, and version declarations for Java 17 and Spring AI `1.0.0`.

Now, let's look at the generated code of your new Spring AI project.

`SpringBootApplication`

Your SpringAI project contains a simple Spring Boot application:

```
@SpringBootApplication
public class ExampleOneApplication {
  public static void main(String[] args) {
    SpringApplication.run(ExampleOneApplication.class);
  }
}
```

We will change the `main` method to use the `SpringApplicationBuilder` class in order to set `WebApplicationType` to `NONE` before the Spring AI application is bootstrapped.

By default, Spring AI transitively pulls in the `spring-web` dependency, which triggers Spring Boot's auto-configuration to start a web server. However, since we are only building a simple **command-line (CLI)** application to begin with, we do not need to start an embedded (Tomcat-based) Servlet or (Netty-based) Reactive web server.

Our modified `main` method now appears as follows:

```
public static void main(String[] args) {
  new SpringApplicationBuilder(ExampleOneApplication.class)
    .web(WebApplicationType.NONE)
    .build()
    .run(args);
}
```

Using Spring Boot's `SpringApplicationBuilder` class is recommended since it gives you more control over the configuration of the Spring Boot application on startup. You can

specify Spring profiles, additional System properties, or explicitly include other Spring `@Configuration`, or `@SpringBootConfiguration`, classes that can be bootstrapped on startup, among other useful functions.

The heart of this Spring AI application is encapsulated in the Spring Boot `ApplicationRunner`, declared as a Spring bean, which prompts the AI and outputs the generated answer, as shown here:

```
@Bean
ApplicationRunner programRunner(ChatModel chatModel) {
 return args -> {
  String prompt = "In a single word, 'what is the answer to life, the universe, and everything?'";
  System.out.printf("User> %s%n%n", prompt);
  String response = chatModel.call(prompt);
  System.out.printf("AI> %s%n%n", response);
 };
}
```

NOTE

Spring Boot's `ApplicationRunner` and `CommandLineRunner` interfaces are virtually interchangeable. The main difference is that the `ApplicationRunner`'s `run(..)` method accepts an `ApplicationArguments` object. This gives users more control over application arguments and command-line options passed to the Java program on startup. In contrast, the `CommandLineRunner's run(..)` method simply accepts a variable argument String array containing the Java program's arguments as passed to the `public static main` method, as is. In other words, by declaring an `ApplicationRunner`, the `ApplicationArguments` makes program arguments conveniently accessible and distinguishes plain arguments from CLI options. Personally, I prefer the `ApplicationRunner`.

Spring AI's `ChatModel` is used to interface with OpenAI. However, if you try to run the program now, it throws an exception:

Caused by: `java.lang.IllegalArgumentException`: OpenAI API key must be set. Use the connection property: `spring.ai.openai.api-key` or `spring.ai.openai.chat.api-key` property.

Spring AI instructs us to set the `spring.ai.openai.api-key` property (or alternatively, the `spring.ai.openai.chat.api-key` property) to the API key we set up after creating an account (https://auth.openai.com/create-account) with OpenAI.

TIP

Follow OpenAI's Developer quickstart guide (https://platform.openai.com/docs/quickstart) from their documentation to get set up to access OpenAI's provided models.

It is recommended that you *do not* declare the `spring.ai.openai.api-key` property in Spring Boot `application.properties` *nor* set your API key as an environment variable. Remember, this is your personal and private API key. It was created for you to give you access to AI models provided by OpenAI. Do not share your API key with anyone or check your API key into source control, such as GitHub, where it is publicly accessible and visible.

If you decide to declare your OpenAI API key in Spring Boot `application.properties`, then I suggest that you follow these steps:

Create a user-specific properties file, such as `application-user.properties`, and set the `spring.ai.openai.api-key` property to your OpenAI API key

Then, enable the **user** profile with - `Dspring.profiles.active=user`

Finally, add `application-user.properties` to your `.gitignore` file or equivalent exclude file used by your source control management (SCM) tool

After configuring your OpenAI API key, re-run the application. You should now see the following output:

> User> In a single word, 'what is the answer to life, the universe, and everything?'
> AI> 42

NOTE

As a further safeguard, it is recommended that you create and use project-based API keys and set limits on `[ monthly|project ]` usage.

Well, I'll be damned! You don't say. I don't know about you, but I never really bought into what Mahatma Gandhi famously said, "*If it's on the Internet it must be true*," but how can you doubt a thing with "*Intelligence*" in its name?

We just learned something very valuable here, besides the secret to life, the universe, and, well, "*everything*"! More importantly, we learned how to shape the generated response of an AI.

The key to the question in our Prompt was, "*In a single word...*". As a result, OpenAI's ChatGPT model responded with **42**. Now that is pretty cool!

We'll learn more about Prompts, and more generally, Prompt Engineering, in **Chapter 2**.

NOTE

See the complete code for this example in GitHub: https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/example-one

---

In the next section we will talk about how you can switch between different AI providers and models.

Switching AI provider models in a single step

We will first start with switching to one of the OpenAI models available.

Switching AI Models

I was deliberately not specific about which OpenAI model we were using.

As you may be aware, OpenAI offers a wide range of models (https://platform.openai.com/docs/models) for different purposes: chat, image generation, audio transcription, or speech to text, text to speech, and so on. In some cases, the models are multimodal meaning they accept multiple forms of input and even generate multiple forms of output, or content.

The default OpenAI model used by Spring AI is `gpt-4o-mini`.

You can control which OpenAI model your application uses by setting the `spring.ai.openai.chat.options.model` Spring AI property to any one of OpenAI's models, such as [`gpt-4o`, `gpt-4o-mini`, `gpt-4-turbo`, ...].

NOTE

See OpenAI's model page (https://platform.openai.com/docs/models) for more information.

For example, you can set the chat options Spring AI model property to dall-e-3 if you want to generate images:

```
spring.ai.openai.chat.options.model=dall-e-3
```

Unlike your OpenAI API key, the model property is not sensitive.

Spring AI properties vary significantly by use case (audio, video, etc.), which we will cover in more detail in **Chapter 2**.

Of course, you may want to use a model from a completely different AI provider altogether.

Switching AI providers

Not all AI providers are equal, even when they host the same (or similar) models. There could be several reasons to change the AI provider, like different model capabilities, costs, latency, factual correctness, multi-modality, among many others.

For example, perhaps you want to use Google Gemini (https://gemini.google.com/app). In this case, simply change your dependency from the OpenAI starter to the Google VertexAI Gemini starter, like so:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>
    spring-ai-starter-model-vertex-ai-gemini
  </artifactId>
</dependency>
```

Once again, dependency version is not required.

Or, perhaps you want to use Amazon's Bedrock AI (https://aws.amazon.com/bedrock/)? Then simply declare:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>
    spring-ai-starter-model-bedrock
  </artifactId>
</dependency>
```

Declaring specific models from different AI providers will require you to set different properties in order to configure and use the model.

For example, to use Google Gemini, you will need to set both the `spring.ai.vertex.ai.gemini.projectId` and `spring.ai.vertex.ai.gemini.location` Spring AI properties in Spring Boot `application.properties` or an equivalent source.

In turn, this requires you to set up a Google Cloud account and create a new project to which you will associate your use of Google Gemini. Follow the instructions given in Spring AI's documentation here, https://docs.spring.io/spring-ai/reference/api/chat/vertexai-gemini-chat.html#_prerequisites, to get started. Do not worry, though, this is only required on initial setup.

NOTE

To use Google's Vertex AI, see the complete list of available locations here, https://cloud.google.com/vertex-ai/docs/general/locations#available-regions.

When we run the application again using Google Gemini, we see similar but different output:

> User> In a single word, 'what is the answer to life, the universe, and everything?'
> AI> Forty-two.

The amount of configuration required varies from one AI provider to another. In any case, you always have a wide array of choices when using Spring AI.

Now, let's see how we can build and run this example entirely from your local machine in the next section.

Using Ollama: Think global, build and run locally

If you are offline, you have an option to run AI locally, on your personal computer, with the help of Ollama (https://ollama.com/). You can conveniently download, install, and run Ollama on Linux, Mac or Windows (https://ollama.com/download).

After you have installed Ollama, you need to choose which AI model to run. There are a wide variety of AI models to choose from as listed here, https://ollama.com/library. Some models are even multimodal, like LLaVa (https://ollama.com/library/llava). You can also find the available AI models on Ollama's official GitHub project page, https://github.com/ollama/ollama?tab=readme-ov-file#model-library.

NOTE

To see all the available software surrounding the Ollama ecosystem, visit the Ollama GitHub organization (https://github.com/ollama). Ollama provides support

for Python and JavaScript users as well.

Open a command-line prompt or terminal on your computer, then run:

```
$ ollama run llama3.2
```

Alternatively, you could enter `ollama run mistral` on the command line if you want to run the Mistral AI model.

NOTE

See llama3.2 documentation, https://ollama.com/library/llama3.2, for more details.

Now, as we did when switching between different AI provider models, we can switch to running our Spring AI application using a local AI model with Ollama by declaring a dependency on the Spring AI's Ollama Starter:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>
    spring-ai-starter-model-llama
  </artifactId>
</dependency>
```

By default, Spring AI expects the Mistral AI model to be used. However, you can configure your choice by setting the `spring.ai.ollama.chat.options.model` Spring AI property.

Here is an example:

```
spring.ai.ollama.chat.options.model=llama3.2
```

Once more, you should see output similar to the following:

```
User> In a single word, 'what is the answer to life, the universe, and everything?'
AI> 42.
```

Congratulations!! You are now running with no strings attached and you have earned a few extra leafs on your Google Nest. In other words, by simply changing the dependency

from OpenAI to Ollama, you are running your Spring AI, Spring Boot application completely and entirely from localhost. No code changes were necessary.

Summary

In this chapter, we learned how to get started quickly by going to the Spring Initializer at https://start.spring.io (where all magic on the Internet happens) to create a project and begin building smart applications in Java using Spring AI.

We started with a simple Spring Boot application using Spring AI to prompt OpenAI's ChatGPT, using OpenAI's `gpt-4o` model. We got back a generated response and output the content to standard out.

Next, we saw how we could easily switch to using different AI models provided by an AI provider along with how to switch between different AI providers, including Google Gemini and Amazon Bedrock.

Finally, we explored how to run an AI locally, on our personal computer, using Spring AI with Ollama. In Chapters *2* and *3* we will explore the capabilities and feature set of Spring AI in more depth.

Get this book's PDF version and more

Scan the QR code (or go to https://packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.





Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

Previous chapter

< Part 1: AI From the Beginning

Next chapter

Chapter 2: Understanding Spring AI: Part 1 >